

## Losses and objectives (Run100 / Run123): CE + CRPS + tail-weighting + (optional) Sharpe

This section documents the four objective components used in the Run100/Run123 training path:

1. **Cross-entropy (CE)** on an ordinal return histogram
2. **Discrete CRPS** to improve distributional calibration
3. **Tail-weighting** applied per token to emphasize large-magnitude moves
4. **Optional Sharpe-style utility term** to directly bias the predictive mean toward trading PnL

The key design choice is that we train a **full predictive distribution** over next-period log returns (via bins), and downstream policy consumes the **distribution-derived expectation**  $\mu = E[r]$ , not a point-regression head.

---

### 0) Setup: distributional head and notation

At each time step/token  $t$ , the model outputs **logits** over  $K$  return bins:

- $\text{logits}[t] \in \mathbb{R}^K$
- $p[t] = \text{softmax}(\text{logits}[t])$ , so  $p[t, k] \geq 0$  and  $\sum_k p[t, k] = 1$

Targets include:

- $y[t]$ : a **target distribution** over bins (soft labels), shape  $\mathbb{R}^K$
- $r[t]$ : the **target realized log return** (scalar)
- $c[k]$ : **bin centers**, derived from bin edges

Masking: losses are computed only on tokens where `target_bin >= 0` (and, in some modes, split masks restrict to train tokens).

---

### 1) Loss 1 — Cross-entropy (CE) on return bins

**Goal:** Train the model to allocate probability mass to the correct region of the return space, using the provided soft target distribution.

Given predicted probabilities  $p[t]$  and target distribution  $y[t]$ :

$$L_{\text{ce}}(t) = - \sum_{k=1}^K y[t, k] \log(p[t, k])$$

**Implementation:**

- `train/losses.py::cross_entropy_from_probs(logits, target_probs)` returns per-token CE.

#### Notes:

- Because  $y[t]$  is a distribution (not just an index), CE acts as a stable “distribution matching” objective and typically trains faster than pure CRPS.
- 

## 2) Loss 2 — Discrete CRPS

**Goal:** Improve **distribution quality** (calibration and sharpness) beyond “put mass in the right bin.” CRPS is a proper scoring rule for distributions on a 1D outcome and penalizes miscalibration across the full support.

For a discrete distribution with support at bin centers  $c[k]$  and predicted probabilities  $p[t,k]$ , with target value  $r[t]$ , CRPS can be written as:

$$\text{CRPS}(p, r) = \mathbb{E}|X - r| - \frac{1}{2} \mathbb{E}|X - X'| \quad \text{where } X, X' \sim p$$

#### Implementation:

- `train/losses.py::crps_discrete(pred_probs, bin_centers, target_values)` computes:
  - `term1 = sum_k p_k * |c_k - r|`
  - `pairwise = sum_{i,j} p_i p_j |c_i - c_j|`
  - returns `term1 - 0.5 * pairwise`

#### Notes:

- CRPS explicitly encourages the *shape* of the predicted distribution to be consistent with the realized numeric return, not just the “correct class.”
  - The pairwise term introduces an  $O(K^2)$  component per token (via the pairwise distance matrix). This is acceptable when  $K$  is modest, but it is materially heavier than CE.
- 

## 3) Loss 3 — Tail-weighting (per-token reweighting)

**Goal:** Allocate more learning capacity to rare, high-impact outcomes (large absolute moves), where PnL variance typically concentrates.

In `scripts/run100_fullseq_llm.py`, each token loss is multiplied by a weight  $w(t)$  based on the absolute realized return magnitude:

```
abs_ret = |target_log_return|  
w = (1 + 10.0 * abs_ret) * (1 + 2.0 * I(abs_ret > 0.03))
```

Equivalently:

- Linear ramp:  $1 + \alpha|r|$  with  $\alpha = 10.0$

- Large-move jump: multiply by  $(1 + \beta)$  when  $|r| > \tau$ , with  $\beta = 2.0$ ,  $\tau = 0.03$

### How it is applied

Tail-weighting is applied to **both CE and CRPS** terms as a **weighted average** over valid tokens:

$$L_{ce} = \frac{\sum_t w(t), L_{ce}(t)}{\sum_t w(t)}, \quad L_{crps} = \frac{\sum_t w(t), L_{crps}(t)}{\sum_t w(t)}$$

### Important nuance: “proper scoring” vs. tail-weighting

- **CE and CRPS are proper scoring rules** when used unweighted (or with constant weights).
- **Outcome-dependent weighting** (weight depends on realized  $r[t]$ ) effectively changes the training emphasis toward tails; it is a deliberate bias toward tail fidelity rather than uniform calibration under the raw empirical distribution.

This is intentional: we are explicitly trading some “average-case” pressure for better performance on high-impact regimes.

---

## 4) Loss 4 — Optional Sharpe-style utility term (off by default)

**Goal:** Provide a direct bridge from distributional forecasting to **trading utility** by encouraging the predicted mean to align with realized returns in a Sharpe-like way.

First compute predicted mean return per token:

$$\mu(t) = \sum_{k=1}^K p[t, k] \cdot c[k]$$

Define “pnl per token” as:

$$\text{pnl}(t) = \mu(t) \cdot r(t)$$

Then compute a differentiable Sharpe proxy over the set of valid tokens:

$$\text{sharpe} = \frac{\mathbb{E}[\text{pnl}]}{\sqrt{\mathbb{E}[\text{pnl}^2] + \varepsilon}} \quad \Rightarrow \quad L_{\text{sharpe}} = -\text{sharpe}$$

**Implementation:** Enabled only when `--lambda_sharpe > 0` in `scripts/run100_fullseq_llm.py`.

**Important:** In the current implementation, **tail-weighting is not applied to the Sharpe term**; only CE/CRPS are weighted.

**Why it is off by default**

- It can destabilize distributional training (especially early), because it pushes `mu` aggressively without necessarily preserving a well-calibrated histogram.
- It optimizes a batch-level utility proxy, which can fight token-level distributional scoring signals.

---

## 5) Full objective (what is actually optimized)

With weights  $\lambda_{ce} = ce\_weight$ ,  $\lambda_{crps} = crps\_weight$ ,  $\lambda_{sh} = lambda\_sharpe$ , the effective loss is:

$$L = \lambda_{ce} L_{ce} + \lambda_{crps} L_{crps} + \lambda_{sh} L_{sharpe}$$

Where  $L_{ce}$  and  $L_{crps}$  are **tail-weighted token averages**, and  $L_{sharpe}$  is an optional unweighted batch statistic over `pnl`.

---

## 6) Why this objective

### 6.1 Distribution first (not IC first)

- **CE + CRPS** trains a *probabilistic forecast*, not a ranker.
- This is aligned with the architecture: the head is a histogram, and downstream computations (e.g., `mu`) are derived from the full distribution.

### 6.2 Calibration matters because policy consumes an expectation

Downstream policy uses:

$$\mu[t] = \sum_k p[t, k] \cdot c[k]$$

So the quality of `p[t]` matters materially:

- Miscalibrated tails distort `mu` and can create unstable exposures.
- CRPS applies pressure to get the whole distribution shape right, not only the argmax bin.

### 6.3 Tail-weighting is a variance-allocation choice

Most trading variance (and many outsized drawdowns) comes from large-magnitude moves. Tail-weighting:

- increases gradient signal on large moves,
- counteracts “regime dilution” where common small moves dominate mini-batch averages.

### 6.4 Sharpe term is a utility bridge, but risky

The Sharpe-style term is a deliberate “policy-aware” objective:

- It encourages  $\mu$  to become a better trading signal.
  - It is kept off by default to avoid collapsing distribution quality into a single-mean objective too early.
- 

## 7) Empirical observations (as reflected in the run docs you cited)

- **CE + CRPS is not an IC objective.** It can improve CE/CRPS without improving rank IC; this is expected because proper scoring rules target calibration, not ordering (see docs/run103\_debug.md).
  - **CRPS-only with high weight can hurt generalization.** Example: run110 notes improved train IC but degraded val/test IC under CRPS-only at high weight (see docs/run110\_run.md).
  - **Mixed CE + CRPS tends to be the stable default.** Multiple runbooks use `loss_mode=ce_crps` as a baseline (e.g., docs/run103a\_run.md, docs/run106\_run.md).
- 

## 8) How the probabilistic head feeds policy (mechanically)

- Model outputs `probs[t,k]` over bins.
- Compute expected return:
  - $\mu[t] = \sum_k \text{probs}[t,k] * \text{bin\_center}[k]$
  - (see `scripts/run125_alpha_policy.py::compute_mu`)
- Policies (top-k, linear-neutral, long/short variants) consume  $\mu$  (and optionally ranks) to form portfolio weights.

This is why we emphasize **distribution quality**:  $\mu$  is not an independent regression head; it is a functional of the entire histogram.

---

## 9) Run123 configuration: what is known vs. unknown

### Closest documented runbook: Run122

docs/run122\_runbook.md is the closest reference. It indicates:

- `loss_mode=ce_crps`
- `ce_weight=1.0`
- `crps_weight=10.0`
- `lambda_sharpe=0.0` (disabled)
- tail weights  $w = (1 + 10*|r|) * (1 + 2*(|r| > 0.03))$
- note: Run122 mentions Run123 experimenting with `gate_init_prob=0.8`,  
but does not document loss changes

### Unknowns that require the original Run123 invocation or checkpoint metadata

- Whether Run123 used Run122's `crps_weight=10.0` vs. script defaults (`crps_weight=1.0`)
  - Whether `lambda_sharpe` was enabled
  - Whether any overrides to `loss_mode` were used
- 

## 10) Implementation caveat (worth calling out explicitly)

If `--use_cuda_graph` is enabled, the current CUDA-graph capture path computes **only an unweighted CE mean** during the captured step (it does not include CRPS or tail-weighting in that code path). If graphs are used in production training, this is a material divergence from the stated objective and should be addressed or disabled.

---

If you want, I can reformat this into your repo's preferred style (e.g., docs/runXYZ\_loss.md) and add a short "TL;DR" header plus a single equation block that matches the exact code behavior (weighted normalization for CE/CRPS; unweighted Sharpe).